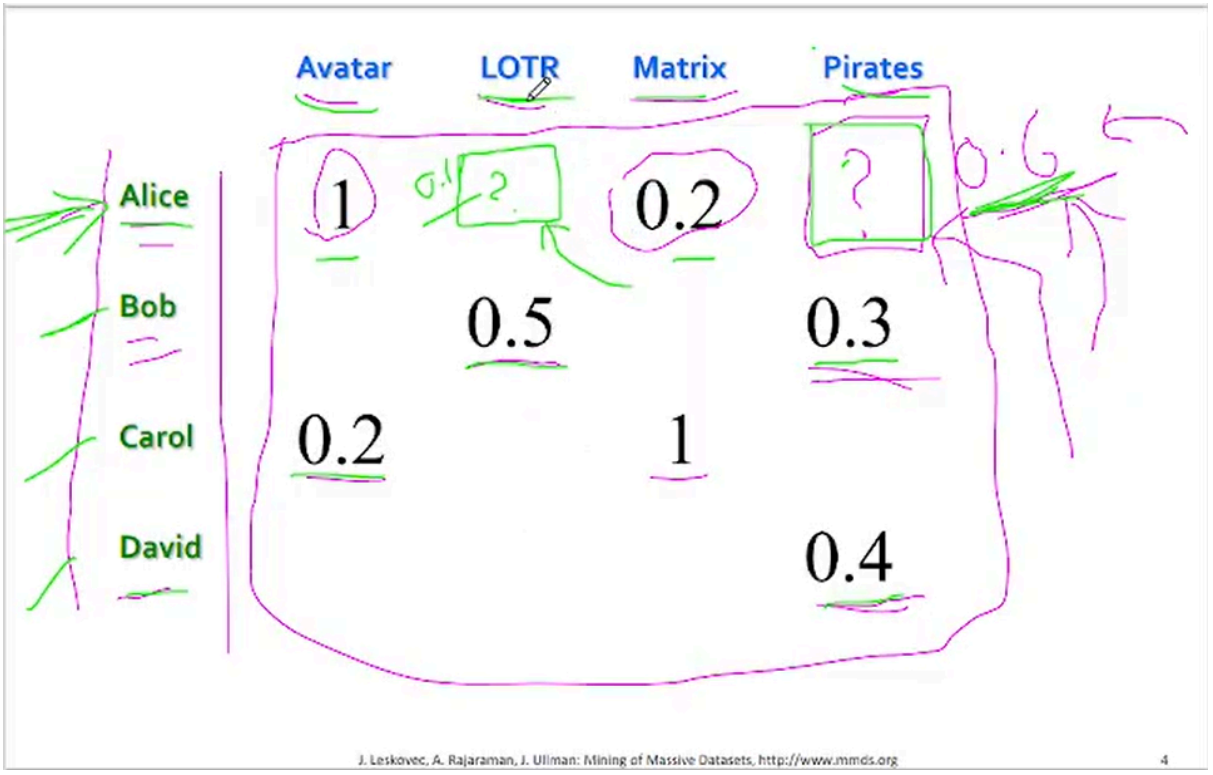
Recommendations

Examples:

amazon.com
PANDORA
StumbleUpon
del.icio.us
NETFLIX
movielens
helping you find the *right* movies
last·fm
the social music revolution
Google News
YouTube
XBOX LIVE

Formal Model

- ***X*** = set of **Customers**
- ***S*** = set of **Items**
- **Utility function** ***u***: ***X*** × ***S*** → ***R***
  - ***R*** = set of ratings
  - ***R*** is a totally ordered set
  - e.g., **0-5** stars, real number in **[0,1]**



Key Problems

- **(1) Gathering "known" ratings for matrix**
  - How to collect the data in the utility matrix
- **(2) Extrapolate unknown ratings from the known ones**
  - Mainly interested in high unknown ratings
    - We are not interested in knowing what you don't like but what you like
- **(3) Evaluating extrapolation methods**
  - How to measure success/performance of recommendation methods

(1) Gathering Ratings

- **Explicit**
  - Ask people to rate items
  - Doesn't work well in practice – people can't be bothered
- **Implicit**
  - Learn ratings from user actions
    - E.g., purchase implies high rating
  - What about low ratings?

(2) Extrapolating Utilities

- **Key problem:** Utility matrix U is **sparse**
  - Most people have not rated most items
  - **Cold start:**
    - New items have no ratings
    - New users have no history
- **Three approaches to recommender systems:**
  1) Content-based
  2) Collaborative
  3) Latent factor based

} Today!

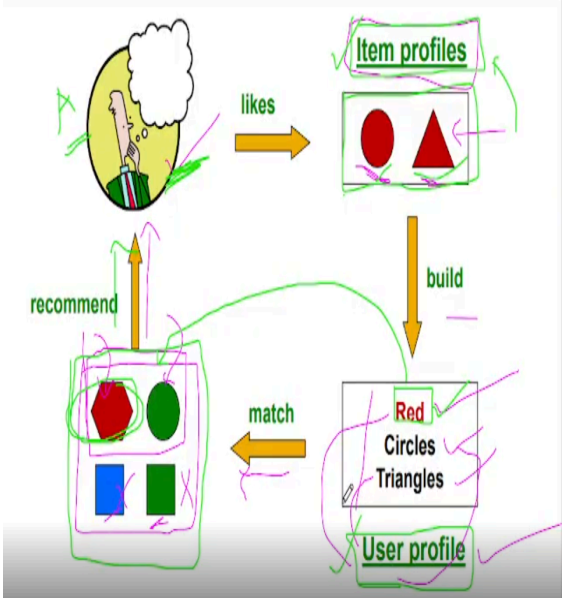
Content-based Recommendations

- **Main idea:** Recommend items to customer **x** similar to previous items rated highly by **x**

Example:

- **Movie recommendations**
  - Recommend movies with same actor(s), director, genre, ...
- **Websites, blogs, news**
  - Recommend other sites with "similar" content

Plan of Action



Item Profiles

- For each item, create an **item profile**
- **Profile is a set (vector) of features**
  - **Movies:** author, title, actor, director,...
  - **Text:** Set of "important" words in document
- **How to pick important features?**
  - Usual heuristic from text mining is **TF-IDF** (Term frequency * Inverse Doc Frequency)
    - Term ... Feature
    - Document ... Item

Sidenote: TF-IDF

f_{ij} = frequency of term (feature) i in doc (item) j

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

Note: we normalize TF to discount for "longer" documents

n_i = number of docs that mention term i

N = total number of docs

$$IDF_i = \log \frac{N}{n_i}$$

TF-IDF score: $w_{ij} = TF_{ij} \times IDF_i$

Doc profile = set of words with highest **TF-IDF** scores, together with their scores

User Profiles and Prediction

- **User profile possibilities:**
  - Weighted average of rated item profiles
  - **Variation:** weight by difference from average rating for item
  - ...
- **Prediction heuristic:**
  - Given user profile $\boldsymbol{x}$ and item profile $\boldsymbol{i}$, estimate

$$u(\boldsymbol{x}, \boldsymbol{i}) = \cos(\boldsymbol{x}, \boldsymbol{i}) = \frac{\boldsymbol{x} \cdot \boldsymbol{i}}{\|\boldsymbol{x}\| \cdot \|\boldsymbol{i}\|}$$

Pros: Content-based Approach

- +: No need for data on other users
  - No cold-start or sparsity problems
- +: Able to recommend to users with unique tastes
- +: Able to recommend new & unpopular items
  - No first-rater problem
- +: Able to provide explanations
  - Can provide explanations of recommended items by listing content-features that caused an item to be recommended

Cons: Content-based Approach

- –: Finding the appropriate features is hard
  - E.g., images, movies, music
- –: Recommendations for new users
  - How to build a user profile?
- –: Overspecialization
  - Never recommends items outside user's content profile
  - People might have multiple interests
  - **Unable to exploit quality judgments of other users**

Based on previous problems (Content Based Cons:) a new system was introduced…

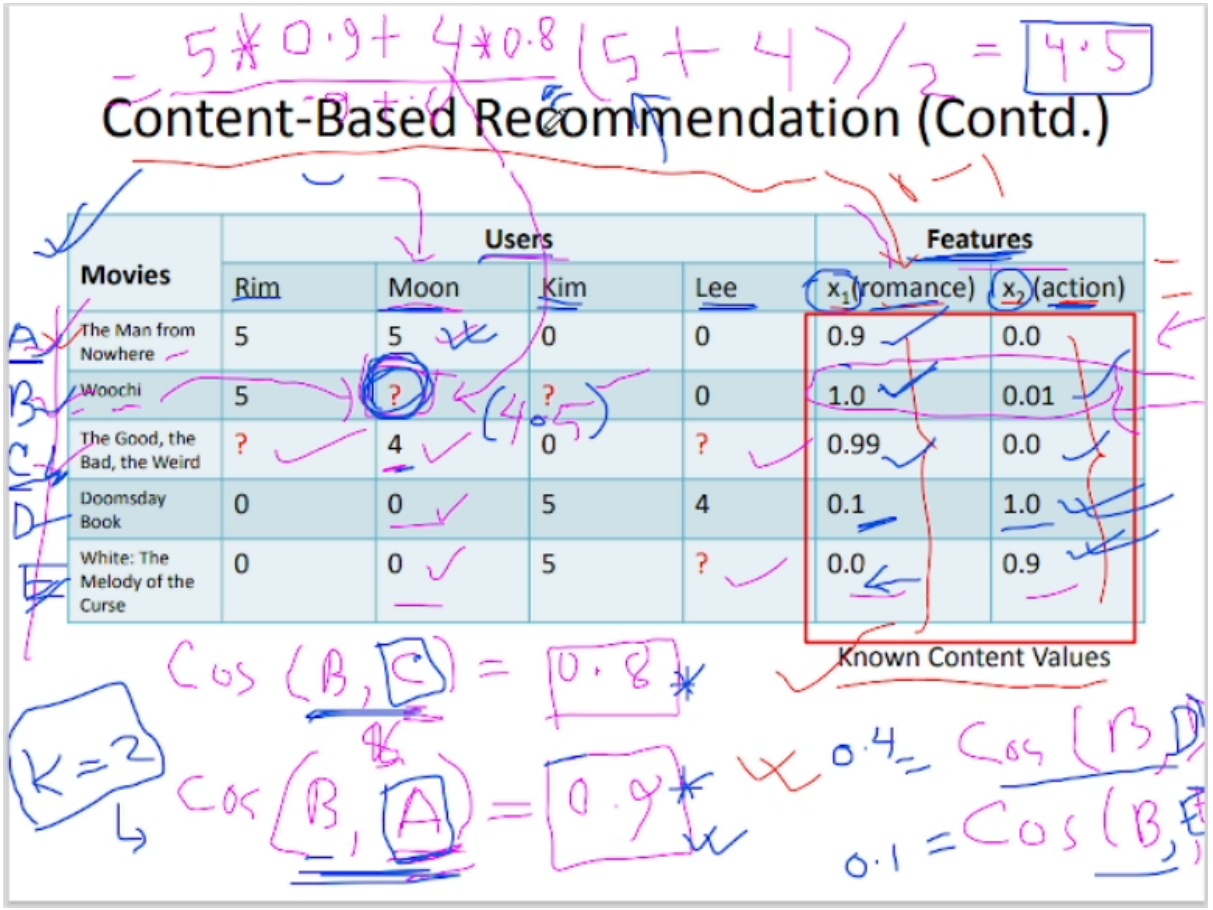
Collaborative Filtering

Harnessing quality judgments of other users

Collaborative Filtering

- Consider user ***x***
- Find set ***N*** of other users whose ratings are "**similar**" to ***x***'s ratings
- Estimate ***x***'s ratings based on ratings of users in ***N***

To be continued… (43 minutes done)



Full Calculation in Notes….

To be continued… (1 hour done)

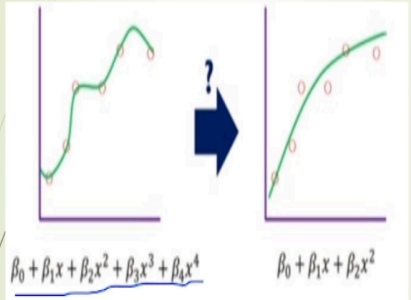
Autonomic M2M service management system

Movies	Users				Features	
	Rim	Moon	Kim	Lee	x_1(romance)	x_2 (action)
The Man from Nowhere	5	5	0	0	0.9	0.0
Woochi	5	?	?	0	1.0	0.01
The Good, the Bad, the Weird	?	4	0	?	0.99	0.0
Doomsday Book	0	0	5	4	0.1	1.0
White: The Melody of the Curse	0	0	5	?	0.0	0.9

Known Content Values

$$\theta_{Moon} = \begin{bmatrix}\theta_0\\ \theta_1\\ \theta_2\end{bmatrix} = \begin{bmatrix}0\\ 5\\ 0\end{bmatrix} \quad X_{Woochi} = \begin{bmatrix}x_0\\ x_1\\ x_2\end{bmatrix} = \begin{bmatrix}1\\ 1.0\\ 0.01\end{bmatrix} \quad \theta^T X = 5.0$$

Regularization



$\beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 x^4$ $\qquad$ $\beta_0 + \beta_1 x + \beta_2 x^2$

$$\min_{\beta} \sum_{i=1} (y_i - \hat{y}_i)^2 + 5000\beta_3^{\,2} + 5000\beta_4^{\,2}$$

$\beta_3 \approx 0 \qquad \beta_4 \not\approx 0$

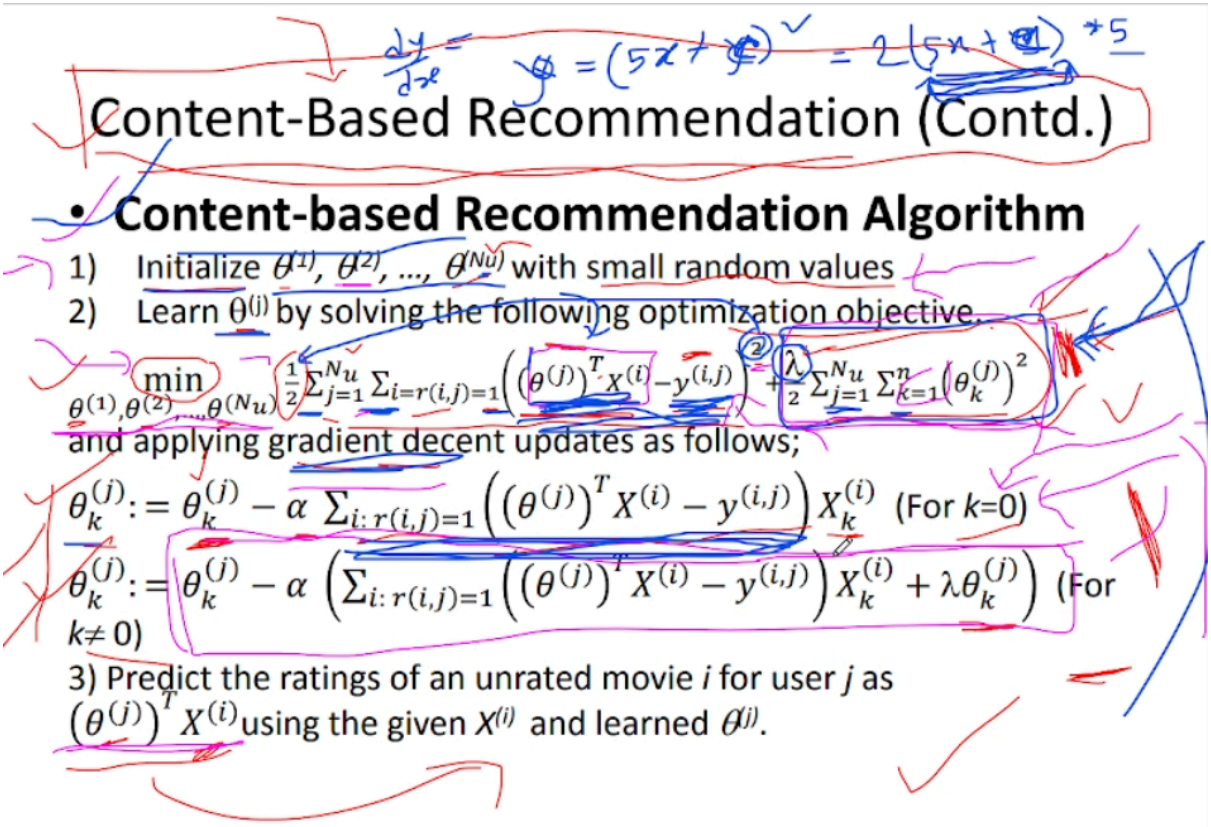
Ridge Regression

$$\beta_1, \beta_2, \ldots, \beta_p$$

$$L(\beta) = \min_{\beta} \sum_{i=1} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

(1) Training accuracy $\qquad$ (2) Generalization accuracy

λ : regularization parameter that controls the tradeoff between (1) and (2)



Some Calculation in Notes (Just Calculation no Implementation)….

To be continued… (1 hour 36 minutes done)

Collaborative Filtering

- Consider user ***x***
- Find set ***N*** of other users whose ratings are "**similar**" to ***x***'s ratings
- Estimate ***x***'s ratings based on ratings of users in ***N***

Finding "Similar" Users

$r_x = [*, \_, \_, *, ***]$
$r_y = [*, \_, **, **, \_]$

- Let r_x be the vector of user ***x***'s ratings
- **Jaccard similarity measure**
  - **Problem:** Ignores the value of the rating
- **Cosine similarity measure**
  - $\text{sim}(x, y) = \cos(r_x, r_y) = \frac{r_x \cdot r_y}{||r_x|| \cdot ||r_y||}$
  - **Problem:** Treats missing ratings as "negative"
- **Pearson correlation coefficient**
  - S_{xy} = items rated by both users ***x*** and ***y***

$$sim(x, y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})(r_{ys} - \overline{r_y})}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \overline{r_y})^2}}$$

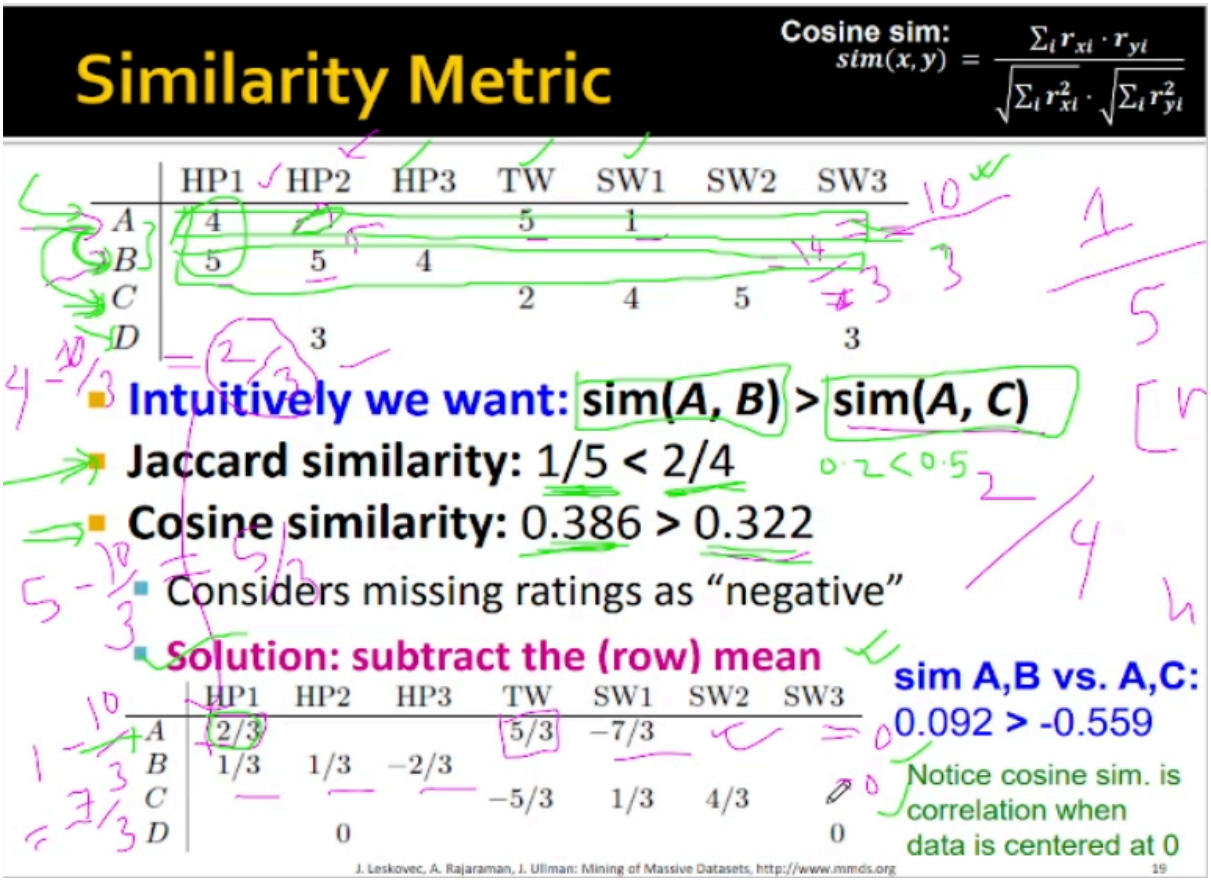
$\overline{r_x}$, $\overline{r_y}$ … avg. rating of **x**, **y**

r_x, r_y as sets:
$r_x = \{1, 4, 5\}$
$r_y = \{1, 3, 4\}$

r_x, r_y as points:
$r_x = \{1, 0, 0, 1, 3\}$
$r_y = \{1, 0, 2, 2, 0\}$

Similarity Metric

Cosine sim: $sim(x, y) = \frac{\sum_i r_{xi} \cdot r_{yi}}{\sqrt{\sum_i r_{xi}^2} \cdot \sqrt{\sum_i r_{yi}^2}}$



Full Math:
https://drive.google.com/file/d/1EOuoIEqEc3jzO4awQFNNegK_sQ727mz6/view?usp=sharing

5️⃣ sim(A, B)

Common item: HP1

Dot product

$$(2/3)(1/3) = 2/9$$

Norms

$$\|A\| = \sqrt{(2/3)^2 + (5/3)^2 + (-7/3)^2} = \sqrt{78/9}$$

$$\|B\| = \sqrt{(1/3)^2 + (1/3)^2 + (-2/3)^2} = \sqrt{6/9}$$

Similarity

$$\text{sim}(A, B) = \frac{2/9}{\sqrt{78/9}\sqrt{6/9}} \approx \boxed{0.092}$$

6️⃣ sim(A, C)

Common items: TW, SW1

Dot product

$$(5/3)(-5/3) + (-7/3)(1/3)$$
$$= -25/9 - 7/9 = -32/9$$

Norms

$$\|A\| = \sqrt{78/9}$$
$$\|C\| = \sqrt{(-5/3)^2 + (1/3)^2 + (4/3)^2} = \sqrt{42/9}$$

Similarity

$$\text{sim}(A, C) = \frac{-32/9}{\sqrt{78/9}\sqrt{42/9}} \approx \boxed{-0.559}$$

↓

Rating Predictions

From similarity metric to recommendations:

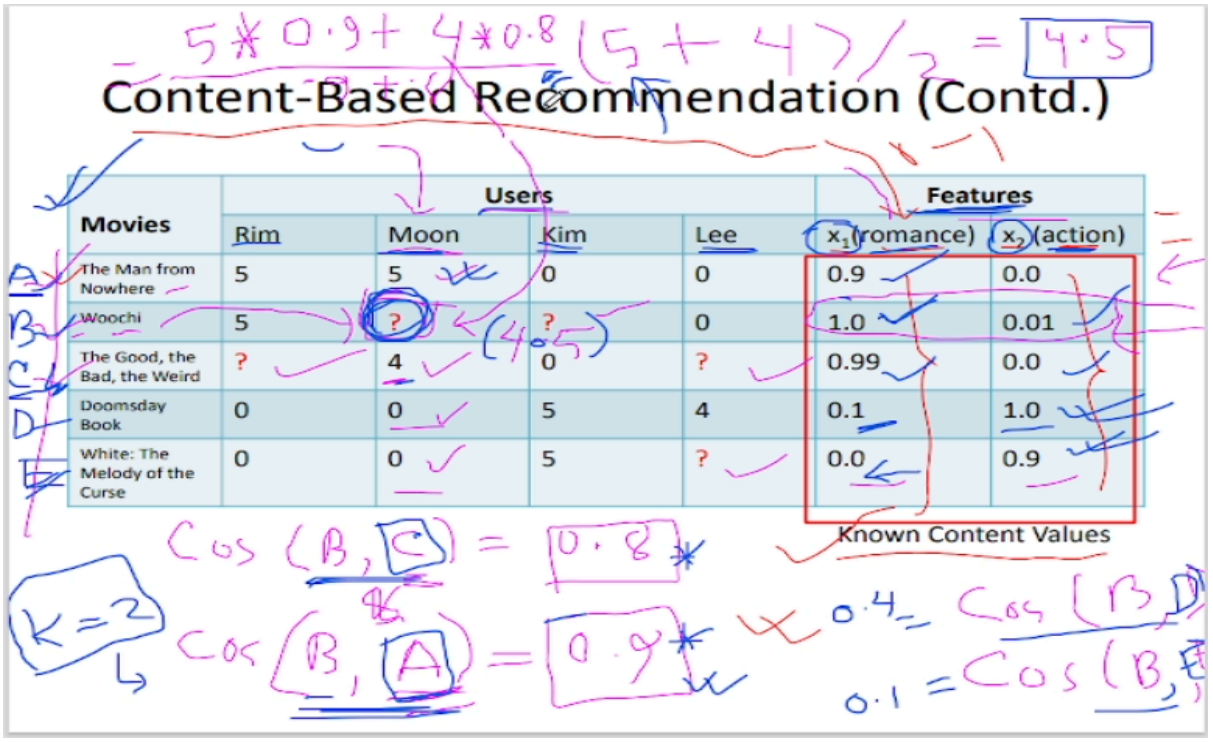
- Let r_x be the vector of user **x**'s ratings
- Let **N** be the set of **k** users most similar to **x** who have rated item **i**
- **Prediction for item s of user x:**
  - $r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$
  - $r_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} s_{xy}}$
  - Other options?
- **Many other tricks possible...**

Shorthand: $s_{xy} = sim(x, y)$

Item-Item Collaborative Filtering

Same as Content Based Collaborative Similarity (See Notes…)

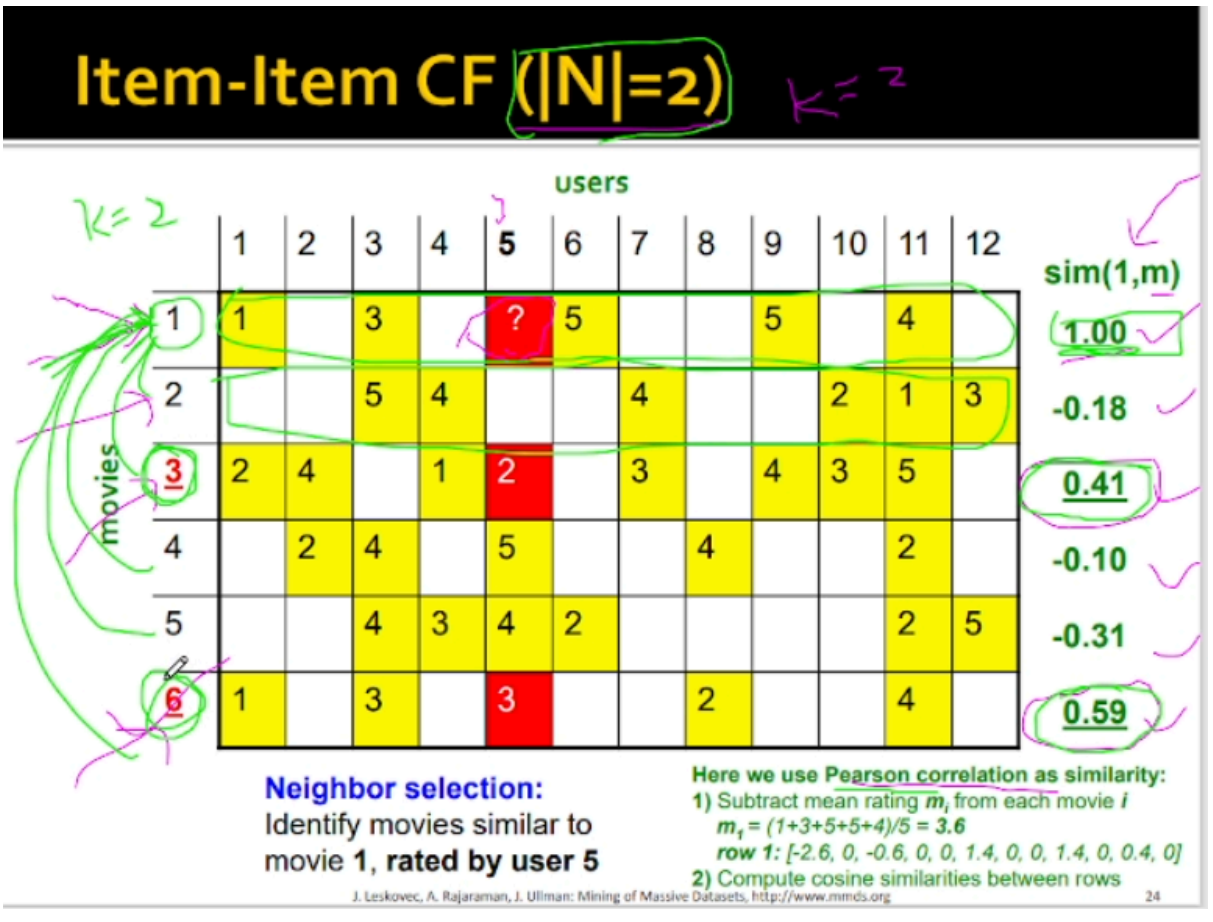
Data Set for Item-Item Collaborative Filtering….



Content-Based Recommendation (Contd.)

Movies	Users				Features	
	Rim	Moon	Kim	Lee	x_1 (romance)	x_2 (action)
The Man from Nowhere	5	5	0	0	0.9	0.0
Woochi	5	?	?	0	1.0	0.01
The Good, the Bad, the Weird	?	4	0	?	0.99	0.0
Doomsday Book	0	0	5	4	0.1	1.0
White: The Melody of the Curse	0	0	5	?	0.0	0.9

Known Content Values



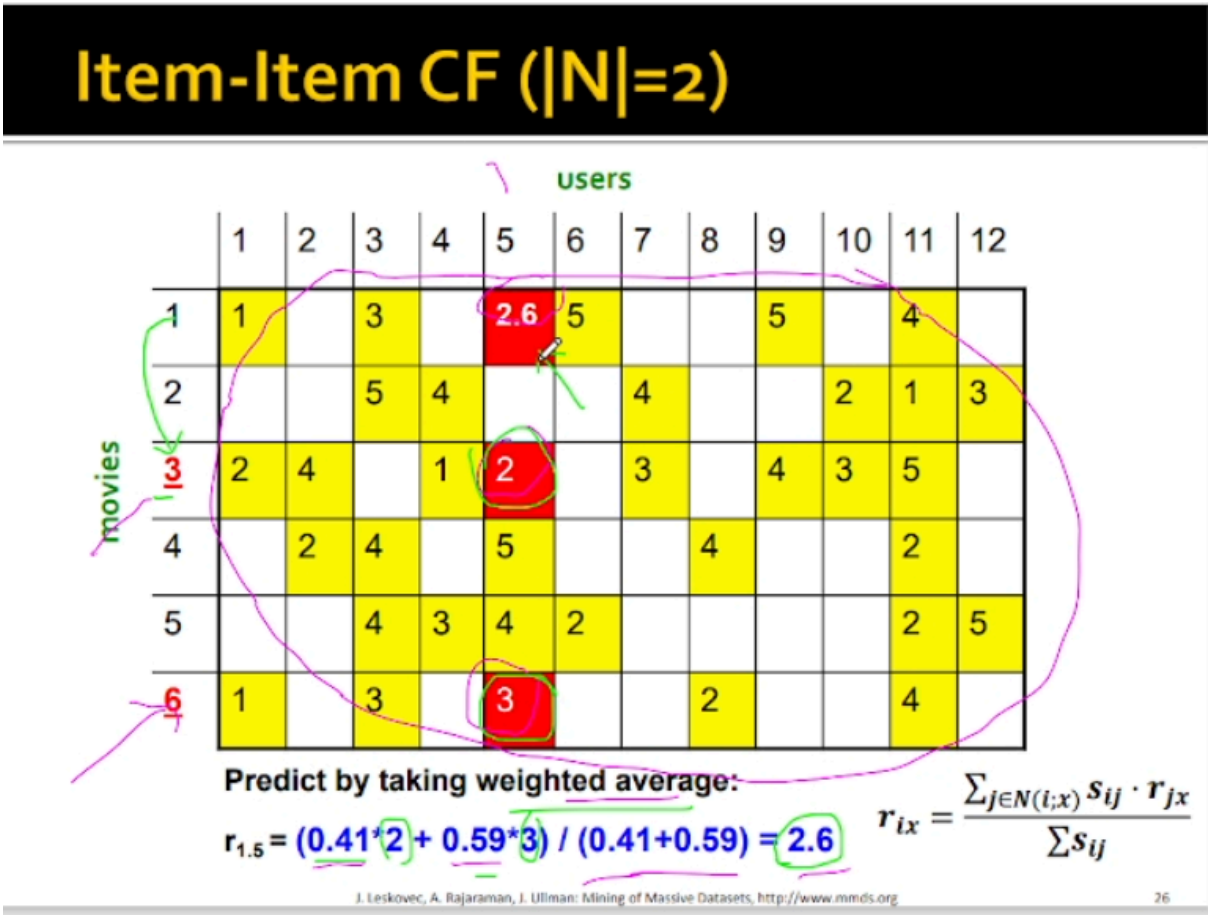
Item-Item CF (|N|=2)

users

movies	1	2	3	4	5	6	7	8	9	10	11	12	sim(1,m)
1	1		3		?	5			5		4		1.00
2			5	4			4			2	1	3	-0.18
3	2	4		1	2		3		4	3	5		0.41
4		2	4		5			4			2		-0.10
5			4	3	4	2					2	5	-0.31
6	1		3		3			2			4		0.59

Neighbor selection:
Identify movies similar to movie **1**, **rated by user 5**

Here we use Pearson correlation as similarity:
1) Subtract mean rating m_i from each movie i
$m_1 = (1+3+5+5+4)/5 = 3.6$
row 1: [-2.6, 0, -0.6, 0, 0, 1.4, 0, 0, 1.4, 0, 0.4, 0]
2) Compute cosine similarities between rows



🔢 Problem recap (from the image)

- Method: **Item–Item Collaborative Filtering**
- Neighborhood size: **|N| = 2**
- Target: **Predict the rating of *User 5* for *Movie 1*** (the red `?`)
- Similarity measure: **Pearson correlation**
- Given similarities with Movie 1:

Movie	sim(1, m)
3	**0.41**
6	**0.59**

↓

➡️ These are the **top 2 neighbors** (highest similarities).

👩‍💻 User 5's ratings for neighbor movies

From the table:

Movie	User 5 rating
3	2
6	3

🧮 Prediction formula (Item–Item CF)

$$\hat{r}_{u,i} = \frac{\sum_{m \in N(i)} \text{sim}(i,m) \cdot r_{u,m}}{\sum_{m \in N(i)} |\text{sim}(i,m)|}$$

🖍️ Plug in the values

Numerator

$$(0.41 \times 2) + (0.59 \times 3) = 0.82 + 1.77 = 2.59$$

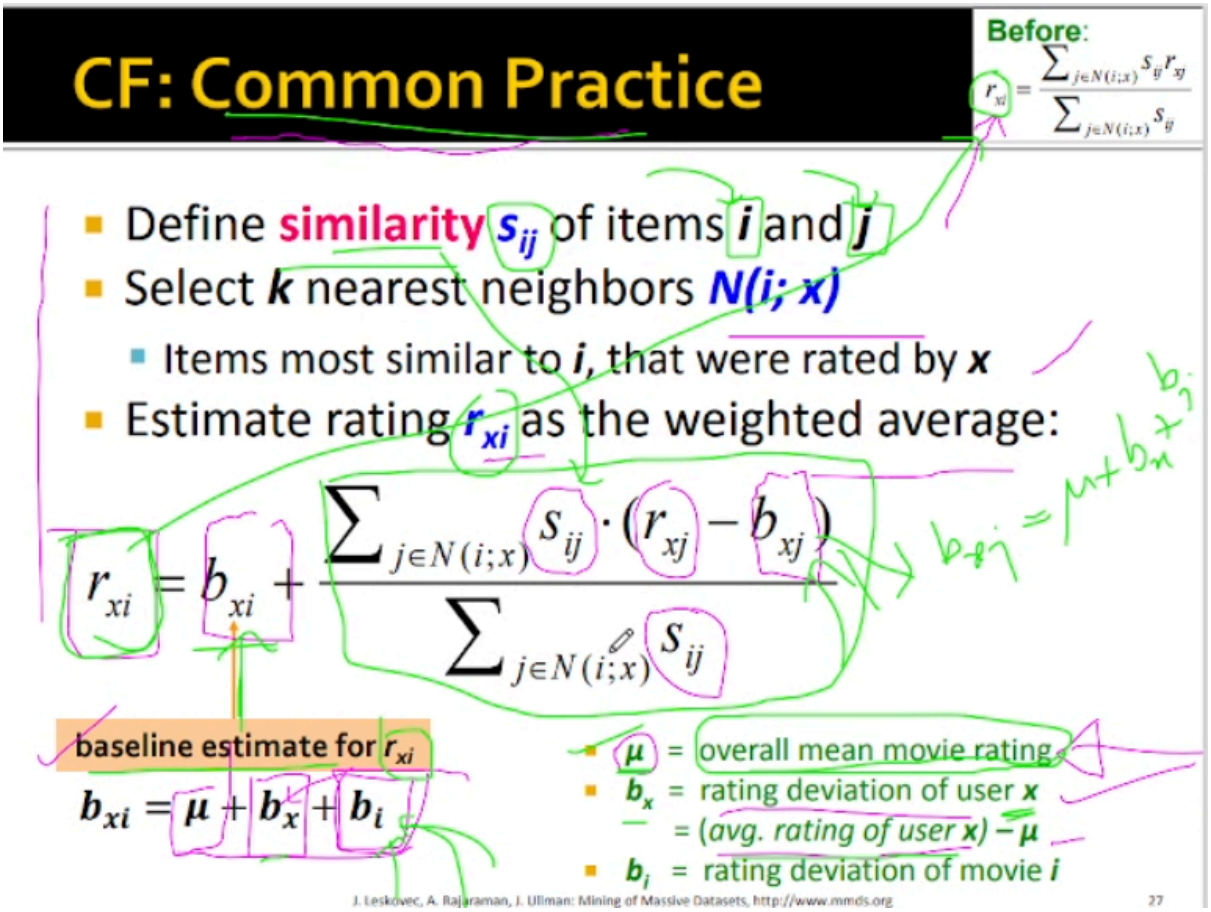
Denominator

$$|0.41| + |0.59| = 1.00$$

✅ Final Answer

$$\boxed{\hat{r}_{5,1} = 2.59 \approx 2.6}$$

📌 **Predicted rating of User 5 for Movie 1 ≈ 2.6**



🔁 Mapping the formula to the previous example

🎯 Goal (same as before)

Predict the rating:

$$r_{5,1}$$

→ **User 5** for **Movie 1**

1️⃣ Identify each symbol in YOUR example

📌 Item and user

- $x = 5$ (user 5)
- $i = 1$ (movie 1)

2️⃣ Neighbors $N(i;x)$

From before:

- Top-2 similar movies to **Movie 1**
- Rated by **User 5**

$$N(1;5) = \{3, 6\}$$

with:

- $s_{1,3} = 0.41$
- $s_{1,6} = 0.59$

3️⃣ Baseline estimate b_{xi}

Formula:

$$b_{xi} = \mu + b_x + b_i$$

What it means in words:

> "What rating would I expect User 5 to give Movie 1 **before** using similarity?"

In **your earlier solution**, we **did NOT use baselines**, so implicitly:

$$b_{xi} = 0$$

📌 That's why the formula reduced to a **simple weighted average**.

4️⃣ Baseline-adjusted neighbor ratings

The general formula uses:

$$(r_{xj} - b_{xj})$$

In your example:

- No baseline was provided
- So:

$$b_{xj} = 0 \Rightarrow (r_{xj} - b_{xj}) = r_{xj}$$

5️⃣ Plug into the full formula

General formula:

$$r_{xi} = b_{xi} + \frac{\sum_{j \in N(i;x)} s_{ij}(r_{xj} - b_{xj})}{\sum_{j \in N(i;x)} s_{ij}}$$

Your example becomes:

$$r_{5,1} = 0 + \frac{0.41(2) + 0.59(3)}{0.41 + 0.59}$$

6️⃣ Final result (same as before)

$$r_{5,1} = \frac{2.59}{1.0} = 2.59 \approx 2.6$$

✅ Same answer

✅ Now explained using the "common practice" formula

🧠 Key insight (VERY exam-important)

- The **simple weighted average formula** is a **special case** of the baseline formula
- It happens when:
  - baselines are **ignored**
  - or assumed to be **zero**

Item-Item vs. User-User

	Avatar	LOTR	Matrix	Pirates
Alice	1		0.8	
Bob		0.5		0.3
Carol	0.9		1	0.8
David			1	0.4

- In practice, it has been observed that item-item often works better than user-user
- **Why?** Items are simpler, users have multiple tastes

Pros/Cons of Collaborative Filtering

- **+ Works for any kind of item**
  - No feature selection needed
- **- Cold Start:**
  - Need enough users in the system to find a match
- **- Sparsity:**
  - The user/ratings matrix is sparse
  - Hard to find users that have rated the same items
- **- First rater:**
  - Cannot recommend an item that has not been previously rated
  - New items, Esoteric items
- **- Popularity bias:**
  - Cannot recommend items to someone with unique taste
  - Tends to recommend popular items

Hybrid Methods

- **Implement two or more different recommenders and combine predictions**
  - Perhaps using a linear model
- **Add content-based methods to collaborative filtering**
  - Item profiles for new item problem
  - Demographics to deal with new user problem